Web Science Application Development

Jason Kuruzovich

Part 2: O

Jason Kuruzovich, PhD
Associate Professor of Business

Feel free to reach out for help

Book a meeting:
https://bit.ly/jason-rpi

Rensselaer Polytechnic Institute
kuruzj@rpi.edu
518-698-9910

Next Parts of Class

Core Engineering Pillars

- **Authentication & Authorization**
- **Testing** (Unit & Integration)
- **Deployment** (Architecture & CI/CD Pipelines)
- **Security Hardening** (DOS/Rate Limiting)
- **Agentic Systems** (LangChain & Tooling)

In Parallel, We Will:

- Deepen our understanding of **React fundamentals and patterns**
- Compare and evaluate **modern web frameworks** (e.g., Flask)

Authentication Learning Objectives

- By the end of class, students should be able to:
- Explain the difference between **Authentication vs Authorization**
- Describe **row-level security (RLS)** and why it matters
- Implement basic email/password auth with Supabase
- Enable OAuth login (Google)
- Enforce per-user data isolation using RLS
- Explain tradeoffs between JWT/session auth and OAuth

Readings

Industry Standards & Best Practice Guides

- 1. OWASP Authorization Cheat Sheet
- 2. WorkOS — OAuth and JWT Best Practices
- 3. Google Developers — OAuth2 Best Practices
- 4. Supabase Authentication

What is the difference between authentication and authorization?

Authentication vs Authorization

- Authentication = Who are you?
- Authorization = What are you allowed to do?

Example:

- Login to Netflix (auth)
- Only see your profile & watch history (authorization)

Authorization: Identify and protect critical system actions

- Examples:
- Create purchase order
- Add vendor
- Issue refund
- Change pricing
- Modify user roles
- Export customer data

These require:

- Strong authentication
- Explicit authorization rules
- Often additional controls (logging, approvals, MFA)

Core Concept: Identity

- User logs in → system issues token (JWT or session)
  - Token sent with every request
  - Backend verifies token
  - Database may enforce identity (RLS)

What Is a JWT?

- **JSON Web Token (JWT):** a compact, URL-safe token for transmitting claims
- Structure:
- **Header** — algorithm & token type
- **Payload** — claims (user ID, role)
- **Signature** — cryptographically signed data
-

- Verified by server without storing session state

How do we securely store passwords?

Password Hashing with bcrypt

- **Never store passwords in plain text!**
- Hashing = one-way function (cannot reverse)
  - bcrypt adds a random salt automatically
  - bcrypt is intentionally slow (cost factor)
- **Flow:**
  - Registration: password → bcrypt.hash() → store hash
  - Login: password + stored hash → bcrypt.compare() → true/false
- **Why not SHA-256?**
  - Too fast — attackers can brute-force billions/sec
  - No built-in salt — vulnerable to rainbow tables

Access Tokens vs Refresh Tokens

- **Access Token**
  - Short-lived (15 min – 1 hr)
  - Sent with every API request
  - Contains user claims (ID, role)
  - If stolen, limited damage window
- **Refresh Token**
  - Long-lived (days – weeks)
  - Used only to get a new access token
  - Stored securely (httpOnly cookie or server-side)
  - Can be revoked to "log out" a user

Token Refresh Flow

- 1. User logs in → server returns access token + refresh token
- 2. Client sends access token with each API request
- 3. Access token expires (e.g., after 15 minutes)
- 4. Client sends refresh token to /api/refresh
- 5. Server verifies refresh token, issues new access token
- 6. If refresh token is expired/revoked → user must log in again
- **Benefits:**
  - Users stay logged in without re-entering credentials
  - Compromised access tokens expire quickly
  - Server can revoke refresh tokens (unlike stateless JWTs)

How does OAuth fit in?

OAuth 2.0: Delegated Authentication

- "Let users log in with Google/GitHub instead of a password"
- **Key Players:**
  - Resource Owner — the user
  - Client — your app
  - Authorization Server — Google, GitHub, etc.
  - Resource Server — API that holds user data
- **Flow (Authorization Code):**
  - App redirects user to provider's login page
  - User approves → provider sends auth code to your app
  - App exchanges code for access token (server-to-server)
  - App uses token to fetch user profile

JWT vs Session-Based Auth

- **Session-Based:**
  - Server stores session in memory/database
  - Client gets a session ID cookie
  - Server looks up session on each request
  - Easy to revoke (delete the session)
- **JWT-Based:**
  - Server does NOT store state — token is self-contained
  - Client stores token (localStorage, cookie, memory)
  - Server verifies signature on each request
  - Hard to revoke before expiry (need blocklist)
- **Trade-off: JWTs scale better; sessions revoke easier**

Middleware: Protecting Routes

- Middleware = function that runs before route handler
- **Authentication middleware pattern:**
  - Extract token from Authorization header
  - Verify token signature and expiration
  - Attach user info to request object
  - Call next() to proceed, or return 401/403
- **401 Unauthorized — no token or invalid credentials**
- **403 Forbidden — token present but invalid/expired**
- Apply per-route: app.get('/secret', authMiddleware, handler)

Security Best Practices

- **Passwords:**
  - Always hash with bcrypt (cost factor >= 10)
  - Never log or return passwords in API responses
- **Tokens:**
  - Keep access tokens short-lived
  - Use HTTPS in production (tokens travel in headers)
  - Store secrets in environment variables, not code
- **General:**
  - Validate all input (username, password length)
  - Rate-limit login attempts to prevent brute force
  - Use generic error messages ("Invalid username or password")

Row-Level Security (RLS)

- Problem: even authenticated users shouldn't see other users' data
- **RLS = database enforces per-user data isolation**
- How it works:
  - Database policy checks user ID on every query
  - SELECT * FROM todos → only returns YOUR todos
  - Even if app code has a bug, DB blocks unauthorized access
- Supported by: Supabase (Postgres), Oracle, SQL Server
- **Defense in depth — don't rely solely on app-level checks**

Lab 6: JWT Authentication Hands-On

- Build a REST API with Express + JWT from scratch:
  - Set up Express server with health check
  - Implement user registration with bcrypt
  - Implement login that returns access + refresh tokens
  - Create middleware to protect routes
  - Add refresh token endpoint
  - Test everything with curl
- **Automated grading: push to GitHub → 8 tests run automatically**
  - Check results in the Actions tab

How do we know our code actually works?

Types of Tests

- **Unit Tests**
  - Test a single function in isolation
  - Example: does bcrypt.hash() produce a valid hash?
  - Fast — run hundreds in seconds
- **Integration Tests**
  - Test components working together
  - Example: does POST /api/register actually store a user?
  - Slower — may need a running server or database
- **End-to-End (E2E) Tests**
  - Test the full user flow from client to server
  - Example: register → login → access protected route
  - Slowest — but catches real-world bugs

The Testing Pyramid

- Many unit tests (fast, cheap, isolated)
- Fewer integration tests (moderate speed, test contracts)
- Few E2E tests (slow, expensive, high confidence)
- **Why a pyramid?**
  - Unit tests catch most bugs quickly
  - Integration tests verify components connect correctly
  - E2E tests confirm the system works as a whole
- **Anti-pattern: "ice cream cone" — too many E2E, too few unit**

Testing in This Lab

- **Our autograder uses integration tests via curl**
  - Each test sends an HTTP request and checks the response
  - Tests run sequentially: register → login → use token
  - This is exactly how you'd test a real API
- **What the 8 tests cover:**
  - Health check (server is running)
  - Registration (happy path + duplicate handling)
  - Login (valid credentials + wrong password)
  - Protected route (no token, valid token, garbage token)
- **Key idea: tests document expected behavior**
  - If the tests pass, the API contract is met

Automated Testing & CI/CD

- **CI = Continuous Integration**
  - Every push triggers automated tests
  - Catch bugs before they reach production
  - GitHub Actions runs our autograder on every push
- **CD = Continuous Deployment**
  - If tests pass, deploy automatically
  - If tests fail, block the deployment
- **Why this matters:**
  - Manual testing doesn't scale
  - Automated tests give confidence to ship fast
  - Tests are a safety net — not a replacement for good design